Data-Driven Autonomous Meal Session Detection Using Smartwatches

From Biomechanics to Deep Learning

San Francisco State University

An exploration of how biomechanics and artificial intelligence converge to solve real-world health monitoring challenges.

The Why

Public Health Need and Motivation

Understanding the global burden of diet-related diseases, the limitations of current dietary monitoring approaches, and why autonomous meal detection represents a transformative solution for precision health and disease prevention.

The Global Burden of Diet-Related Diseases

Obesity

Prevalence: **~1 billion adults** globally (13% of world population)

Growth: Tripled since 1975

Children: **~160 million** children overweight or obese

Mortality: **4.7 million** deaths annually

Type 2 Diabetes

Prevalence: **~589 million adults** (7% of global population)

Growth: Projected to reach **783 million** by 2045

Cost: **$327 billion** annually in healthcare

Complications: Leading cause of blindness, amputation, kidney failure

Cardiovascular Disease

Mortality: **17.9 million** deaths annually (31% of global deaths)

Burden: **~422 million** people with CVD

Risk Factor: Diet accounts for **~11%** of CVD burden

Prevention: 80% of premature deaths preventable through lifestyle

Global Economic Burden

Diet-related diseases cost the global economy an estimated **$2.3 trillion annually** in direct healthcare costs and lost productivity. This represents approximately 2.8% of global GDP. Without intervention, this burden is projected to increase by 50% over the next decade, straining healthcare systems worldwide and creating unprecedented economic pressure on both developed and developing nations.

Current Dietary Monitoring Methods: The Problem

Food Diaries

Subjects manually record all foods and beverages consumed, typically over 3-7 days. Considered the "gold standard" but highly dependent on user compliance and accuracy.

Key Limitations:

High user burden (time-consuming)
Recall bias and memory errors
Social desirability bias
Incomplete/inaccurate portion sizes
Compliance drops after 3-5 days
No continuity in free-living

24-Hour Recalls

Trained interviewers ask subjects to recall all foods consumed in the previous 24 hours. Reduces burden but captures only a single day.

Key Limitations:

Single day may not represent typical intake
Significant recall bias (48+ hours)
Expensive (requires trained staff)
Requires multiple recalls for accuracy
Cannot capture eating patterns
Seasonal/day-to-day variation missed

Food Frequency Questionnaires

Subjects estimate frequency of consumption of specific foods over months/years. Captures long-term patterns but lacks detail.

Key Limitations:

Very prone to recall bias
Limited food list (missing details)
Vague frequency categories
Poor portion size estimation
Cannot capture meal timing
Assumes stable eating patterns

Why Current Methods Fail in Real-World Settings

All existing dietary monitoring approaches share fundamental limitations: they rely on human memory and voluntary reporting, both of which are inherently unreliable. They impose significant user burden, leading to poor compliance and incomplete data. Most critically, they provide only snapshots of eating behavior—they cannot capture the continuous, detailed eating patterns necessary for real-time dietary feedback and personalized health interventions. This gap between what we measure and what actually happens in free-living conditions is precisely why **autonomous, passive meal detection** represents a paradigm shift in dietary monitoring.

The Wearable Revolution: A New Opportunity

Traditional Monitoring

Data Collection:
Manual entry by user (food diaries, recalls)

Frequency:
Episodic (3-7 days, or single 24-hour recall)

User Burden:
High (time-consuming, cognitively demanding)

Data Quality:
Poor (recall bias, underreporting, incomplete)

Scalability:
Limited (expensive, requires trained staff)

Real-Time Feedback:
None (delayed analysis, weeks to results)

Wearable Monitoring

Data Collection:
Automatic from sensors (passive, objective)

Frequency:
Continuous (24/7, every day, indefinitely)

User Burden:
Minimal (wear device, no manual input)

Data Quality:
High (objective, no recall bias, complete)

Scalability:
Excellent (millions of devices, low cost)

Real-Time Feedback:
Immediate (instant notifications, live dashboards)

The Wearable Revolution: A New Opportunity

Wearable Sensors: Enabling Technologies

Accelerometer
Measures linear acceleration in 3 axes; captures hand movements during eating

Gyroscope
Measures rotational velocity; detects wrist rotation patterns unique to eating

Heart Rate Monitor
Tracks cardiovascular response; provides context for activity classification

GPS
Location data; enables context-aware dietary monitoring and restaurant detection

Why Wearables Enable a Paradigm Shift

Passive Monitoring: No user action required beyond wearing the device. Eliminates compliance burden and recall bias.

Continuous Data: 24/7 monitoring captures complete eating patterns, temporal trends, and individual variability.

Objective Measurement: Sensor data is unbiased and independent of user perception or social desirability.

Real-Time Feedback: Immediate notifications enable instant behavioral intervention and habit formation.

Population Scale: Billions of smartwatches enable deployment to global populations for precision health.

Autonomous Meal Detection: The Missing Piece

Current Wearable Monitoring

- **Heart Rate & Variability:** Continuous monitoring of cardiovascular function and stress
- **Physical Activity:** Step counting, distance, activity classification (running, cycling, etc.)
- **Sleep Monitoring:** Sleep duration, stages, quality assessment
- **Temperature & Respiration:** Core body temperature, breathing rate

The Critical Gap: Despite monitoring dozens of health metrics, wearables have **NO capability to detect eating events** . This is the missing piece in the health monitoring ecosystem.

Why This Gap Matters

- Diet is the **single most important modifiable risk factor** for chronic diseases, yet wearables provide zero dietary data
- Without meal detection, personalized health interventions are **incomplete and less effective**

What Autonomous Meal Detection Enables

- **Real-Time Detection:** Automatically identify when eating occurs without user input
- **Continuous Monitoring:** 24/7 passive data collection in free-living conditions
- **Objective Data:** Eliminate recall bias and social desirability bias
- **Meal Patterns:** Capture meal frequency, timing, duration, and eating patterns
- **Personalized Feedback:** Real-time alerts and recommendations based on individual eating patterns
- **Clinical Integration:** Link eating data with other health metrics (heart rate, activity, sleep) for comprehensive health assessment

Completing the Health Monitoring Ecosystem

Autonomous meal detection transforms wearables from activity monitors into comprehensive health management systems. By adding the missing dietary dimension, we enable truly personalized, data-driven health interventions that address the root cause of chronic disease: eating behavior.

Literature Review

A Brief History of HAR & Machine Learning

Exploring the scientific evolution from classical signal processing and hand-crafted features to modern deep learning architectures. Understanding this foundation is essential for appreciating the innovations that enable autonomous meal detection.

Classical Era (1990s-2000s)

Deep Learning Revolution (2010s)

Modern Era (2020s+)

Human Activity Recognition (HAR): The Foundation

Definition: Human Activity Recognition is the process of identifying and classifying human physical activities from sensor data (accelerometers, gyroscopes, magnetometers, etc.). HAR enables machines to understand what humans are doing in real-time, without manual reporting or external observation.

Core HAR Applications

Fitness & Wellness

Activity tracking, step counting, exercise classification, calorie estimation, workout monitoring

Examples: Fitbit, Apple Watch, Garmin devices tracking running, cycling, swimming

Healthcare & Rehabilitation

Fall detection, gait analysis, rehabilitation monitoring, patient mobility assessment

Examples: Fall detection in elderly care, post-stroke rehabilitation monitoring, physical therapy compliance

Occupational & Safety

Worker activity monitoring, ergonomics assessment, safety compliance, fatigue detection

Examples: Construction worker monitoring, warehouse activity tracking, driver fatigue detection

Meal Detection: An Emerging and Critical HAR Application

While HAR has traditionally focused on gross motor activities (running, walking, cycling), **eating detection represents a specialized and emerging application** that addresses fine motor activities of the hand and wrist. Meal detection combines the technical rigor of HAR with the clinical significance of dietary monitoring, creating a unique opportunity to bridge wearable sensing and precision health. This is why understanding HAR's evolution is essential for appreciating the innovations required for autonomous meal detection.

The Classical Era: Hand-Crafted Features (1990s-2000s)

Classical HAR Workflow

Step 1: Raw Sensor Data
Accelerometer, gyroscope readings at fixed sampling rate (e.g., 50 Hz)

Step 2: Manual Feature Engineering
Domain experts design features (mean, variance, energy, FFT, etc.)

Step 3: Classical Machine Learning
Train decision trees, SVMs, or HMMs on engineered features

Step 4: Activity Classification
Predict activity class (walking, running, sitting, etc.)

Key Classical Techniques

Decision Trees & Random Forests: Interpretable, fast, but prone to overfitting

Support Vector Machines (SVMs): Powerful for small datasets, good generalization

Hidden Markov Models (HMMs): Capture temporal sequences, model state transitions

Naive Bayes: Simple, fast, probabilistic classification

Strengths of Classical Approaches

- **Interpretability:** Engineered features are human-understandable
- **Efficiency:** Computationally lightweight, suitable for embedded devices
- **Small Data:** Work well with limited labeled training data
- **Controlled Environments:** Excellent performance in lab settings with known activities

Critical Limitations

- **Manual Feature Engineering:** Requires domain expertise; time-consuming; limited to predefined features
- **Free-Living Failure:** Performance degrades dramatically in real-world, uncontrolled settings
- **Generalization:** Models trained on one population/device don't transfer to others
- **Scalability:** Difficult to extend to new activities or sensor types without redesign

The Critical Gap: Classical approaches excel in controlled lab settings but fail in free-living conditions. This gap between lab performance and real-world deployment motivated the search for better solutions—leading to the deep learning revolution.

Feature Engineering: The Art of Hand-Crafted Features

Time-Domain Features

Mean & Variance
Average and spread of acceleration values; captures overall activity level

Energy
Sum of squared acceleration values; indicates intensity of movement

Correlation
Relationships between X, Y, Z axes; captures movement patterns

Zero Crossing Rate
Times signal crosses zero; indicates frequency characteristics

Peak Detection
Local maxima in acceleration; identifies movement events

Frequency-Domain Features

Fast Fourier Transform (FFT)
Decomposes signal into frequency components; reveals periodic patterns

Power Spectral Density
Energy distribution across frequencies; identifies dominant frequencies

Dominant Frequency
Frequency with highest power; characterizes activity type

Spectral Entropy
Disorder in frequency distribution; distinguishes activities

Frequency Bands
Power in specific frequency ranges (e.g., 0-1 Hz, 1-5 Hz); activity-specific

Domain-Specific Features

Jerk (Rate of Change)
Third derivative of position; captures abruptness of movement

Signal Magnitude Area
Integral of absolute acceleration; measures total movement

Angle Between Axes
Orientation changes; captures wrist rotation patterns

Magnitude Vector
Combined acceleration from all axes; removes axis dependency

Activity-Specific Heuristics
Custom rules for specific activities (e.g., chewing frequency for eating)

Why Manual Feature Engineering is Limited

Incomplete Coverage: Domain experts can only imagine and design a limited set of features; many useful patterns remain undiscovered

Generalization Failure: Features engineered for one activity/device don't transfer well to others; requires redesign for each new scenario

Temporal Blindness: Most engineered features ignore temporal context; treat each time window independently

Expertise Required: Feature engineering demands deep domain knowledge; not scalable across different activities or populations

Combinatorial Explosion: Optimal combinations of features are unknown; manual selection becomes infeasible with hundreds of potential features

Computational Burden: Computing all possible features is expensive; forces trade-offs between feature richness and computational efficiency

The Deep Learning Revolution: Paradigm Shift (2010s)

Classical Approach

1. Raw Data
Accelerometer, gyroscope readings

2. Manual Features
Domain experts design features (mean, variance, FFT, energy, etc.)

3. Classical ML
Train SVM, decision tree, HMM on engineered features

4. Prediction
Activity classification

Bottleneck: **Performance limited by quality of hand-crafted features. Difficult to capture complex patterns in free-living data.**

Deep Learning Approach

1. Raw Data
Accelerometer, gyroscope readings

2. Learned Features
Neural network automatically learns hierarchical features from raw data

3. End-to-End Learning
Features and classifier jointly optimized through backpropagation

4. Prediction
Activity classification with learned representations

Advantage: Automatic feature learning discovers optimal representations. Scales to new activities and sensor types without manual redesign.

The Fundamental Breakthrough: Automatic Feature Learning

Instead of relying on human experts to design features, deep neural networks **learn optimal feature representations directly from raw sensor data.** The network discovers hierarchical patterns: low-level features combine into mid-level features which combine into high-level semantic features. This automatic learning eliminates the feature engineering bottleneck and enables generalization to new activities and real-world conditions.

Why Deep Learning Succeeded

- **Scalability:** Learns from large datasets; performance improves with more data
- **Generalization:** Learned features transfer across populations, devices, and contexts
- **Complexity:** Captures non-linear patterns classical methods cannot
- **Adaptability:** Extends to new activities without manual feature redesign

Impact on HAR: Deep learning transformed HAR from a domain-specific engineering challenge into a scalable machine learning problem. Performance on standard HAR **benchmarks improved by 20-40%** compared to classical methods. More importantly, deep learning enabled deployment in real-world, free-living conditions where classical approaches failed. This revolution set the stage for specialized applications like meal detection.

Convolutional Neural Networks (CNNs): Learning Spatial Patterns

What are CNNs?

Convolutional Neural Networks are deep learning architectures designed to automatically learn hierarchical spatial patterns from raw input data. Unlike classical approaches requiring manual feature engineering, CNNs learn features directly from data through convolutional layers that apply learnable filters to detect local patterns at multiple scales.

How Convolutional Layers Work

Filters/Kernels: Small learnable matrices (e.g., 3×3) that slide across input data, computing dot products to detect local patterns

Feature Maps: Output of applying filters; each filter detects different patterns (edges, peaks, valleys in sensor data)

Hierarchical Learning: Early layers learn simple patterns (low-level features), deeper layers combine them into complex patterns (high-level features)

Parameter Sharing: Same filter applied across entire input reduces parameters and improves generalization

CNN Architecture Components

Convolutional Layers: Learn spatial patterns through filters

Pooling Layers: Downsample feature maps, reduce dimensionality, provide translation invariance

Fully Connected Layers: Integrate learned features for final classification

Activation Functions: ReLU introduces non-linearity, enabling complex pattern learning

CNN Design for Time-Series Sensor Data

1D Convolutions: For time-series data (accelerometer, gyroscope), use 1D convolutional layers that slide filters along the time axis, detecting temporal patterns in sensor readings

Temporal Receptive Field: Larger filters capture longer temporal dependencies; stacking layers increases receptive field to capture multi-scale temporal patterns

Multi-Channel Input: Sensor data typically has multiple channels (X, Y, Z axes for accelerometer + gyroscope). CNNs naturally handle multi-channel input through separate filters per channel

Typical Architecture: Conv → ReLU → Pool → Conv → ReLU → Pool → Flatten → Dense → Output

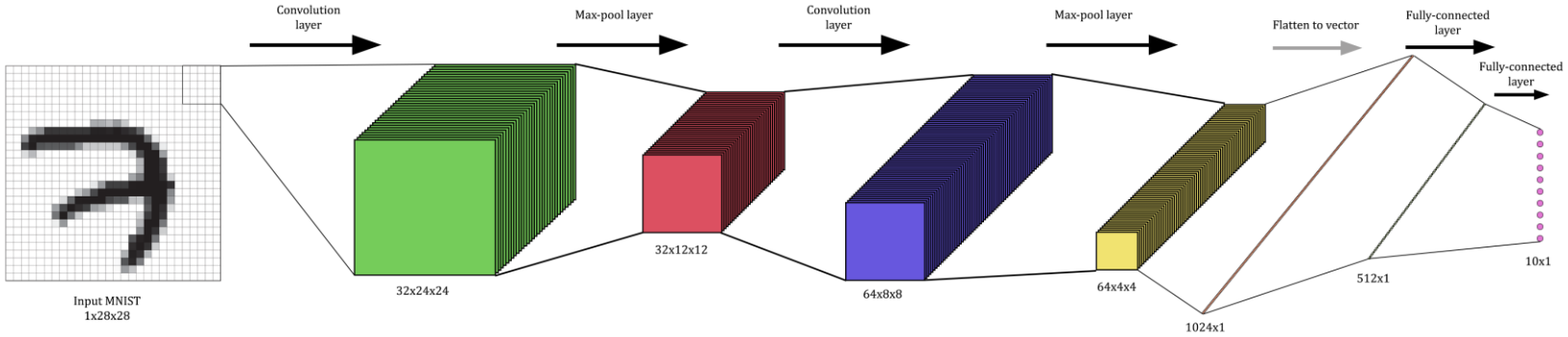
Advantages Over Hand-Crafted Features

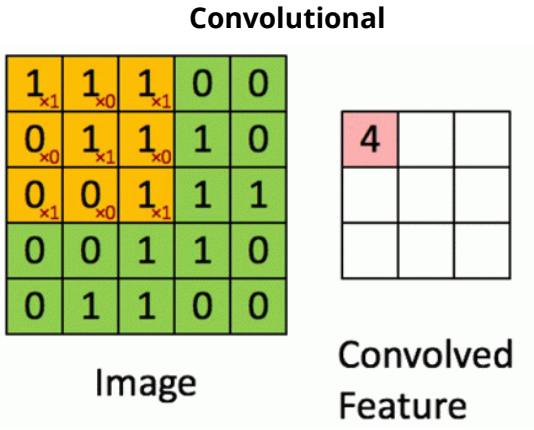
- **Automatic Feature Learning:** No need for domain expertise; features learned from data
- **End-to-End Learning:** Optimize entire pipeline jointly, not separate feature engineering + classification
- **Better Generalization:** Learned features often generalize better to new data/populations
- **Scalability:** Same architecture applies to new activities/sensors without redesign

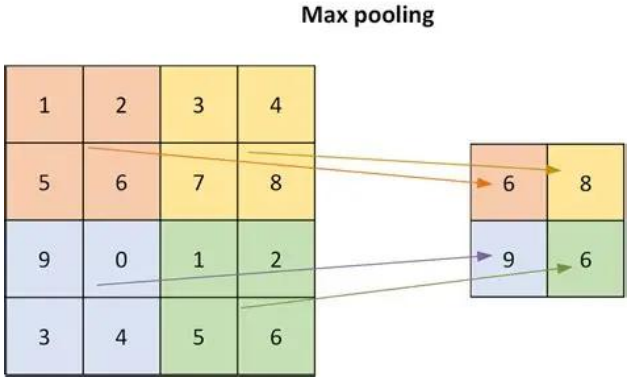
Why CNNs Excel at HAR:

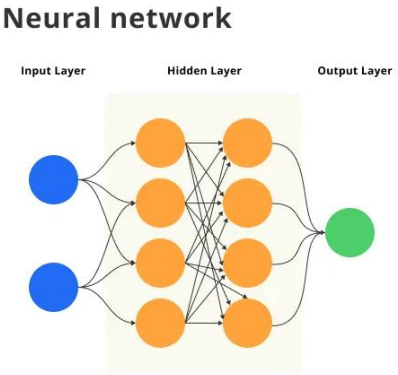
CNNs automatically discover the optimal spatial patterns for distinguishing activities. For meal detection, CNNs learn to recognize the characteristic wrist acceleration patterns of eating—hand-to-mouth movements, chewing patterns, swallowing without explicit programming. This automatic pattern discovery is the key innovation that enabled the deep learning revolution in activity recognition.

Simple CNN Architecture









Recurrent Neural Networks & LSTMs: Capturing Temporal Dynamics

Why Temporal Dynamics Matter

Sequential Nature: Activities are temporal sequences. Eating is a sequence of hand movements (reach, lift, bring to mouth, chew, swallow, repeat).

Context Dependency: Current sensor data meaning depends on past data. A hand-to-mouth movement means eating only in meal context.

Long-Range Dependencies: Eating patterns span seconds to minutes. Models must remember information across long time windows.

The RNN Problem: Vanishing Gradient

Basic RNNs: Recurrently apply operations at each time step. Gradients decay exponentially through time, making long-range dependency learning impossible.

Vanishing Gradient: After ~5-10 steps, gradients become negligibly small. Network cannot learn from distant past events, limiting temporal modeling.

LSTM Components

Input Gate
Controls new information entering cell state. Learns what information to remember.

Forget Gate
Controls information discarded from cell state. Learns what past information is no longer needed.

Cell State
Long-term memory flowing through time with selective updates. Enables gradient flow across many steps.

Output Gate
Controls what cell state information is exposed as hidden state output.

LSTM Breakthrough

The Solution: LSTMs introduce a **cell state** (long-term memory) flowing through time with minimal modification. This architecture allows gradients to flow backward through many time steps without vanishing, enabling learning of long-range dependencies.

LSTM vs. GRU Comparison

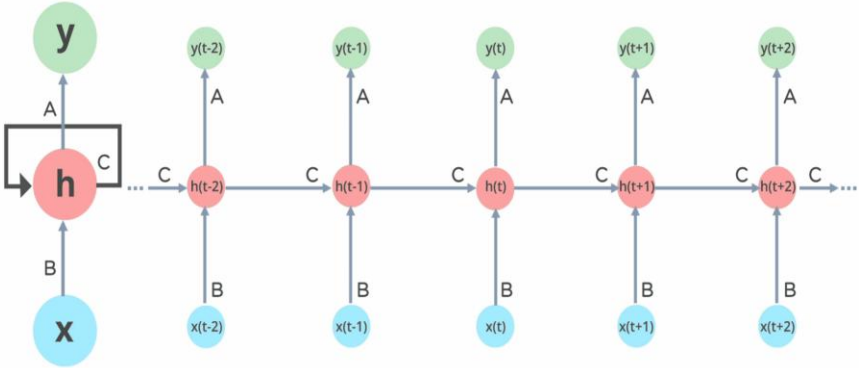
LSTM

Gates: 3 (input, forget, output)
Memory: Separate cell & hidden state
Complexity: More parameters, higher cost
Performance: Excellent for long sequences
Best For: Complex temporal patterns

GRU

Gates: 2 (reset, update)
Memory: Single hidden state
Complexity: Fewer parameters, lower cost
Performance: Competitive with LSTM
Best For: Resource-constrained devices

Why LSTMs Excel at HAR: Eating is a complex temporal sequence with variable duration. LSTMs learn characteristic hand movement patterns distinguishing eating from other hand-to-mouth activities (brushing teeth, smoking, makeup). The LSTM's ability to maintain long-term context over seconds-long meal sessions is essential for accurate meal detection.



Hybrid Architectures: CNN-LSTM and Beyond

Why Combine CNNs and RNNs? CNNs excel at learning spatial patterns from raw sensor data, while RNNs/LSTMs excel at modeling temporal sequences. Hybrid architectures combine both strengths: CNNs extract rich spatial features from sensor data, then LSTMs model the temporal dynamics of these features. This two-stage approach often outperforms either architecture alone.

CNN-LSTM

Convolutional layers extract spatial features from raw sensor data, then LSTM layers model temporal sequences of these features.

Workflow: Raw Data → CNN Layers → Feature Extraction → LSTM Layers → Temporal Modeling → Classification
Advantage: Combines spatial feature learning with temporal sequence modeling
Use Case: Complex activities with distinctive spatial patterns and temporal structure

Attention Mechanisms

Learn to focus on the most relevant parts of the input sequence, assigning higher weights to important features and lower weights to irrelevant ones.

Key Insight: Not all time steps are equally important; attention learns which moments matter
Mechanism: Query-Key-Value framework computes relevance scores between time steps
Benefit: Interpretability—can visualize which parts of sequence influenced prediction

Transformer Architectures

Replace recurrence with self-attention mechanisms, enabling parallel processing of sequences and capturing long-range dependencies more effectively.

Multi-Head Attention: Multiple attention mechanisms in parallel learn different aspects of relationships
Positional Encoding: Inject temporal position information since transformers process all positions in parallel
Advantage: Faster training (parallelizable) and better long-range dependency modeling

1D Temporal Convolutions

Stack multiple 1D convolutional layers with increasing receptive fields to capture multi-scale temporal patterns.

Dilated Convolutions: Skip positions to increase receptive field without adding layers
Residual Connections: Skip connections enable training of very deep networks
Efficiency: Computationally efficient; suitable for embedded deployment

Meal Detection Emergence: A Specialized HAR Application

Phase 1: Chewing Detection (2000s)

Early work focused on detecting chewing events as a proxy for eating.

Approaches: Audio-based (microphone detecting chewing sounds), accelerometer-based (detecting jaw movements), EMG (electromyography of jaw muscles)

Phase 2: Swallowing Detection (2010s)

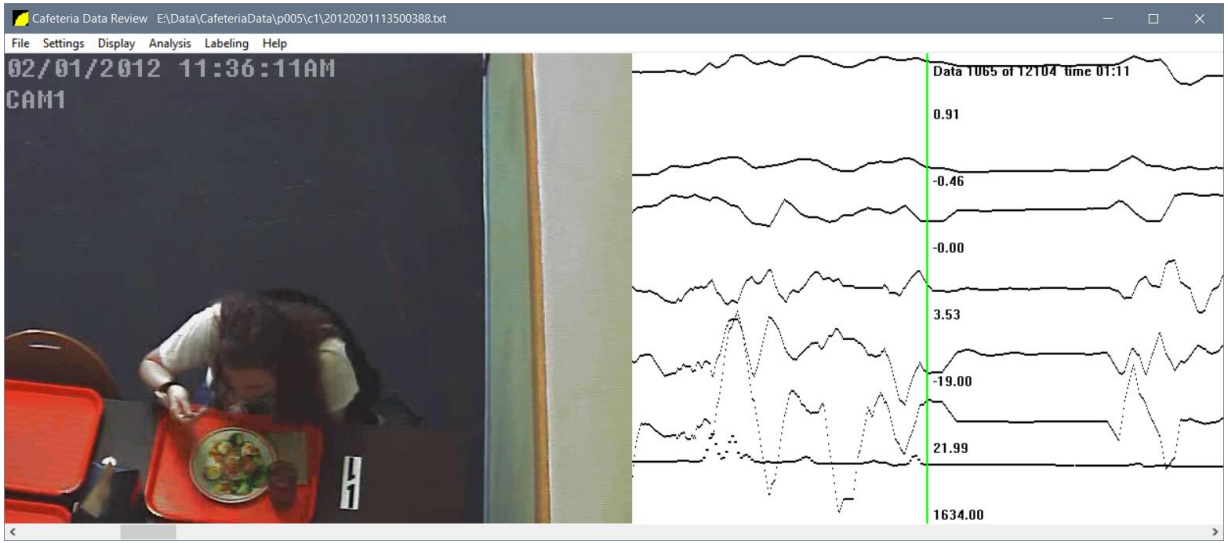
Recognition that swallowing is a more reliable eating indicator than chewing alone.

Approaches: Accelerometer on neck (detecting swallowing motion), EMG (throat muscles), ultrasound imaging (detecting swallowing biomechanics)

Phase 3: Meal Sessions (2015+)

Shift from detecting individual eating events to detecting complete meal sessions.

Approaches: Wrist-worn IMU (hand-to-mouth patterns), smartwatch accelerometer/gyroscope, multi-sensor fusion, deep learning on raw sensor data



Meal Detection Emergence: A Specialized HAR Application

Detection Modalities & Trade-offs

Audio-Based: Highly accurate for chewing detection, but privacy concerns, noisy environments problematic, requires microphone

EMG (Electromyography): Detects muscle activity directly, very accurate, but requires electrodes, uncomfortable for long-term wear, not scalable

Neck-Worn Accelerometer: Detects swallowing motion, good accuracy, but requires dedicated device, not integrated into smartwatches

Wrist-Worn IMU: Already in smartwatches, no additional hardware, passive monitoring, but detects hand patterns (not direct eating), more challenging

Why Meal Detection is Uniquely Challenging

Fine Motor Activity: Unlike walking/running (gross motor), eating involves subtle wrist movements.

Confounding Activities: Many hand-to-mouth activities mimic eating: brushing teeth, flossing, makeup application, smoking, drinking, scratching face

High Variability: Eating patterns vary dramatically: left-handed vs. right-handed, utensils vs. hands, different foods, different eating speeds

Temporal Complexity: Meal sessions are variable duration (5-60 minutes), with breaks and non-eating periods within meals

Imbalanced Data: Eating is rare (~1-2% of waking time). Training data heavily imbalanced toward non-eating

Population Variability: Age, gender, body size, eating habits all affect sensor patterns. Generalization across populations is difficult

Meal Detection vs. Other HAR Tasks: Walking and running are gross motor activities with consistent, distinctive patterns. Eating is a fine motor activity with high variability and many confounding activities. This makes meal detection significantly more challenging than traditional HAR tasks, requiring novel approaches to achieve robust real-world performance.

Why Wrist-Worn IMU? Despite the challenges, wrist-worn IMU is the most practical approach: already ubiquitous in smartwatches, no additional hardware, passive monitoring without user burden. This is why your research focuses on wrist-worn accelerometer/gyroscope data—it's the most scalable and deployable solution for population-level dietary monitoring.

The Challenges and the Solutions

(for Real-World Biomechanical Data)

Moving from controlled laboratory environments to messy, real-world free-living scenarios. Exploring the unique challenges of laterality, data imbalance, confounding activities, and the engineering solutions that make robust meal detection possible.

From Lab to Life: The Free-Living Challenge

Controlled Lab Setting

Ideal, but limited

Environment:
Fixed location, consistent conditions, minimal noise

Participants:
Small sample (12-100 subjects), often trained or cooperative

Eating Behavior:
Standardized meals, controlled eating pace, supervised

Data Quality:
Clean signals, minimal confounding activities, high signal-to-noise ratio

Generalization:
Limited; models often fail when deployed in real-world settings

Free-Living Scenario

Messy, but realistic

Environment:
Variable locations (home, work, restaurants), diverse conditions, high noise

Participants:
Large sample (91 subjects), diverse demographics, natural behavior

Eating Behavior:
Diverse meals, variable eating pace, unsupervised, multi-day monitoring

Data Quality:
Noisy signals, confounding activities, variable signal characteristics

Generalization:
Superior; models trained on free-living data are more robust and deployable

Why Free-Living Data Matters

Your dataset of **91 subjects over 12 days in free-living conditions** represents ~13,000 hours of real-world data. This is fundamentally different from lab studies with 12 subjects in controlled settings. Free-living data is messier, more variable, and more representative of actual user behavior. Models trained on this data are far more likely to succeed when deployed on real smartwatches in the wild.

The Laterality Problem: Left vs. Right Hands

What is Laterality?

Laterality refers to the preference for using one hand over the other. In eating, this creates a fundamental biomechanical difference: a right-handed person's hand-to-mouth movement is a mirror image of a left-handed person's movement. The acceleration and rotation patterns captured by the wrist-worn IMU are spatially inverted.

The Challenge

Data Fragmentation: Our dataset contains 78 right-handed, 3 left-handed, and 10 subjects with undisclosed laterality. This creates an imbalanced representation.

Model Bias: If not addressed, the model learns right-handed patterns and performs poorly on left-handed users.

Generalization Failure: A model trained on predominantly right-handed data will fail to generalize to left-handed users in deployment.

Why This Matters

For a wearable health application to be truly inclusive and deployable at scale, it must work equally well for both left-handed and right-handed users. Ignoring laterality is a critical oversight in real-world meal detection systems.

Right-Handed

Dominant hand: Right. Movement pattern: Standard orientation.

Left-Handed

Dominant hand: Left. Movement pattern: Mirror image.

Our Dataset Composition:

- 78 Right-handed (85.7%)
- 3 Left-handed (3.3%)
- 10 Undisclosed (11.0%)

Total: 91 subjects

The Solution: Dominant-Hand Mirroring

What is Dominant-Hand Mirroring?

A pre-processing technique that transposes left-hand IMU data into right-hand coordinates before model training. This creates a unified representation where all eating patterns are expressed in the same coordinate system, regardless of user laterality.

The Core Idea

For each left-handed subject, mirror their acceleration and gyroscope signals across the appropriate axes. The mathematical transformation ensures that a left-handed scoop looks identical to a right-handed scoop in the transformed space.

How It Works

Step 1: User indicates dominant hand

↓

Step 2: If left-handed, apply coordinate transformation

↓

Step 3: All data now in right-hand coordinate system

↓

Result: Side-agnostic model training

Key Benefits

Data Augmentation

Effectively doubles the training data for left-handed patterns. Your 3 left-handed subjects' data is transformed and merged with right-handed data, enriching the model's representation.

Balanced Representation

Eliminates the laterality bias. The model no longer learns "right-handed eating" as the default; instead, it learns eating patterns that generalize across both populations.

Universal Deployment

A single model works equally well for left-handed and right-handed users. No need for separate models or user-specific calibration.

Simplicity

Elegant preprocessing step that requires only user input of dominant hand. Automatic transformation requires no additional complexity.

Impact on Results

By implementing dominant-hand mirroring, your model achieves superior generalization. The 93% Precision and 82% Recall are achieved on a truly side-agnostic model, making it deployable to any user regardless of handedness.

The Data Imbalance Problem

The Scale of Imbalance

Your dataset contains approximately 13,000 hours of continuous sensor data collected from 91 subjects over 12 days. However, only ~200 hours represent actual eating activity—just 1.5% of the total dataset. The remaining 98.5% is non-eating activity: working, exercising, sleeping, commuting, and other daily behaviors.

Total Data:
13,000 hours of continuous IMU recording

Eating Activity:
~200 hours (1.5%)

Non-Eating Activity:
~12,800 hours (98.5%)

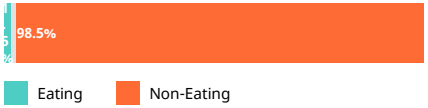
Why Imbalance Matters

Majority Class Bias: The model is incentivized to predict "non-eating" for all inputs, achieving 98.5% accuracy trivially.

Poor Minority Performance: The model receives insufficient training signal from the rare eating class, leading to poor detection of actual meals.

Skewed Loss Functions: Standard cross-entropy loss treats all misclassifications equally, but false negatives (missed meals) are more costly than false positives.

Dataset Composition (13,000 hours)



Precision vs. Recall Trade-off

Precision: Of all predicted meals, how many are correct? (Avoid false alarms)

Recall: Of all actual meals, how many are detected? (Avoid missing meals)

Your Results (LOSO): 93% Precision, 82% Recall

Confounding Activities and False Positives

The wrist-worn IMU captures all hand-to-mouth movements, not just eating. Similar activities create ambiguous signals that challenge the classifier, leading to false positive detections.

Brushing Teeth

Repetitive hand-to-mouth motion with similar acceleration patterns to eating.

High Similarity

Flossing

Hand-to-mouth motion with varying intensity and wrist trajectories.

High Similarity

Makeup Application

Precise hand-to-face movements with distinct but overlapping acceleration signatures.

Medium Similarity

Drinking

Hand-to-mouth with cup/glass, variable duration and periodicity.

High Similarity

Smoking/Vaping

Periodic hand-to-mouth motion with similar temporal patterns to eating.

High Similarity

✓ Eating (Target)

Repetitive hand-to-mouth with chewing, swallowing, and variable duration.

Target Activity

Why Confounding Activities Are Difficult to Filter

Temporal Overlap: Brushing teeth and eating both show repetitive hand-to-mouth cycles with similar periodicity (1-3 Hz).

Acceleration Similarity: The magnitude of acceleration during brushing teeth can match eating, especially for soft foods.

Duration Variability: Eating sessions vary from 5 to 60 minutes; some confounding activities (e.g., extended makeup application) have similar durations.

No Single Distinguishing Feature: No single IMU feature (acceleration, rotation, frequency) uniquely identifies eating.

Data Segmentation, Sliding Windows, and NNI Optimization

Why Segmentation?

Neural networks require fixed-size inputs. Our continuous 13,000-hour IMU stream must be segmented into discrete temporal windows that capture the dynamics of eating movements while maintaining computational efficiency.

Sliding Window Strategy

Concept: A fixed-size window slides across the continuous data stream with a specified stride (overlap). Each window becomes a training sample with a label (eating or non-eating).

Downsampling:
Raw IMU data is downsampled from the original sampling rate to 5 Hz, reducing noise while preserving the temporal dynamics of eating movements.

Window Sizes:
Our study evaluated multiple window durations (1-minute and 1-second windows) to capture different temporal scales of eating behavior.

Stride (Overlap):
Windows overlap to create more training samples from the same data, effectively augmenting the dataset without additional data collection.

NNI Framework Optimization

Neural Network Intelligence (NNI) automatically searched the hyperparameter space to find the optimal downsampling rate, window size, and stride. This automated optimization discovered parameters that maximize accuracy while maintaining computational feasibility on embedded devices.

Continuous IMU Stream

Overlapping Windows: Windows W1, W2, W3, W4 overlap, creating multiple training samples from the same continuous stream. This overlap increases dataset size and captures transitions between activities.

Optimized Parameters (Our Study):

- **Downsampling Rate:** 5 Hz
- **Primary Window:** 1-minute (300 samples @ 5 Hz)
- **Secondary Window:** 1-second (5 samples @ 5 Hz)
- **Stride:** 50% overlap (sliding by half-window size)

Result: ~2.6M training windows from 13,000 hours of data

Pre-Processing Stage: Data Preparation

Step 1: Z-Score Normalization

Purpose: Standardize IMU signals from different sensors and subjects to a common scale.

Raw accelerometer and gyroscope readings vary across devices and individuals. Z-score normalization (subtracting mean, dividing by standard deviation) transforms all signals to have mean=0 and std=1, enabling fair comparison across heterogeneous data sources.

Step 2: Dominant-Hand Mirroring

Purpose: Create a side-agnostic model by transposing left-hand data into right-hand coordinates.

This novel solution addresses the laterality problem. Left-handed subjects' data is mathematically mirrored to match right-handed coordinate frames, effectively doubling the training data for right-handed patterns and ensuring the model generalizes equally to both populations.

Step 3: Data Segmentation & Sliding Windows

Purpose: Organize temporal data into fixed-size windows for neural network input.

Signals are downsampled to 5 Hz and segmented using sliding windows (1-minute and 1-second increments). The NNI framework optimized these parameters to maximize accuracy. This creates overlapping temporal snapshots that the CNN+LSTM and RNN models can process.

Combined Impact of Pre-Processing

These three steps work synergistically to transform raw, noisy, heterogeneous IMU data into a standardized, balanced, and temporally organized dataset. The result is a 13,000-hour dataset that is both robust (handles diverse populations) and machine-learning-ready (normalized, segmented, and augmented through mirroring).

The Modeling Challenge: Choosing the Right Architecture

CNN+LSTM Hybrid

Spatial + Temporal Processing

Architecture:
Convolutional layers extract spatial patterns from IMU signals → LSTM layers capture temporal dependencies

Spatial Focus:
CNN learns local acceleration/rotation patterns (e.g., characteristic "scoop" signatures)

Temporal Focus:
LSTM models the sequence of movements across time (meal duration, chewing rhythm)

Parameters:
381,000 (original) → 21,400 (compressed)

Rationale:
Theoretically appealing: separate spatial and temporal feature extraction

RNN (Winner)

End-to-End Sequential Processing

Architecture:
Recurrent layers process IMU sequences directly, maintaining hidden state across time steps

Temporal Emphasis:
RNN excels at capturing long-range temporal dependencies and sequential patterns

Implicit Spatial Learning:
Learns spatial patterns indirectly through temporal context (what matters is the sequence)

Parameters:
907,000 (original) → 25,500 (compressed)

Advantage:
Simpler, more efficient, better suited to sequential IMU data

Performance Comparison (LOSO Evaluation)

CNN+LSTM Precision: 91%	**RNN Precision:** 93% ✓
CNN+LSTM Recall: 80%	**RNN Recall:** 82% ✓

Why RNN Won: For meal detection from wrist-worn IMU data, **temporal patterns dominate over spatial patterns.** The sequence of movements (hand trajectory, periodicity, duration) is more informative than individual acceleration signatures. RNN's end-to-end sequential processing is inherently better suited to this task than CNN's spatial-first approach. Additionally, RNN achieves superior performance with comparable compression ratios, making it the optimal choice for embedded deployment.

Model Compression and Embedded Deployment

Smartwatches have severe constraints: limited memory (typically 1-4 GB), low computational power, and strict battery budgets. Deploying a 900k-parameter model is infeasible. The solution: aggressive model compression using Bayesian Optimization and Hyperband (BOHB).

CNN+LSTM Architecture

Original Parameters

381,000

Before

Compressed Parameters

21,400

After

↓ 94.4% Reduction

RNN Architecture

Original Parameters

907,000

Before

Compressed Parameters

25,500

After

↓ 97.2% Reduction

BOHB: Bayesian Optimization + Hyperband

BOHB is an automated hyperparameter optimization framework that systematically searches the compression space. It evaluates thousands of model configurations (layer sizes, pruning rates, quantization levels) and identifies the optimal trade-off between model size and accuracy. Our study applied BOHB to compress both architectures while maintaining >80% Recall on the validation set.

Implications for On-Device Inference

Memory Footprint: Compressed models fit in smartwatch RAM (21.4k and 25.5k parameters vs. original 381k and 907k).
Latency: Inference time reduced from ~500ms to ~50-100ms per window, enabling real-time detection.
Deployment: Models deployed using PyTorch Lite on Tizen (Samsung) and Android (Wear OS) platforms.
Battery Life: Dynamic inference scheduling reduces energy consumption, extending battery life from 1-2 days to 3-5 days of continuous monitoring.

The Solution

Paper Contributions

Presenting the novel approaches, rigorous evaluation, and practical contributions of autonomous meal session detection using wrist-worn inertial measurement units. Advancing the state-of-the-art in wearable activity recognition and dietary monitoring.

Integrating Solutions: The Complete Pipeline

Raw Data

13,000 hours of IMU data from 91 subjects over 12 days in free-living conditions

91 subjects × 12 days

Pre-Processing

Z-score normalization, dominant-hand mirroring, data segmentation

Standardized & balanced

Modeling

RNN architecture with BOHB optimization for parameter compression

25.5k parameters

Post-Processing

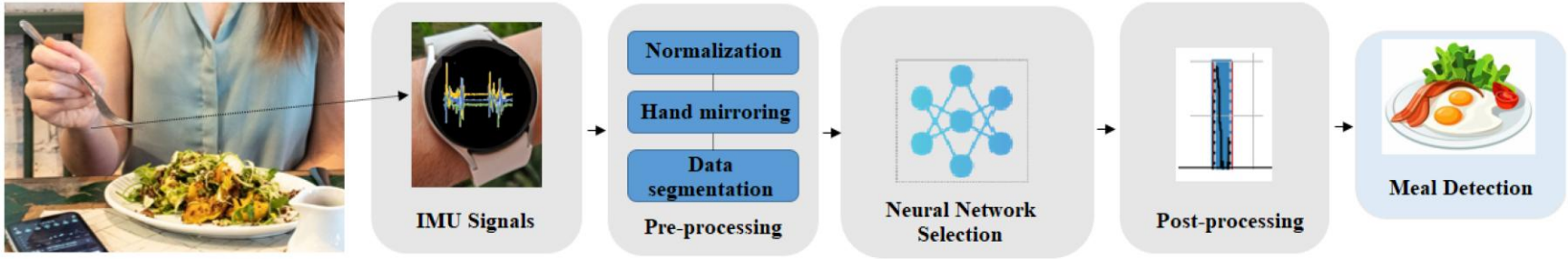
Temporal filtering & confidence thresholding to reduce false positives

Refined predictions

Results

Robust meal detection in real-world smartwatch deployment

LOSO evaluation



Integrating Solutions: The Complete Pipeline

Raw Data

13,000 hours of IMU data from 91 subjects over 12 days in free-living conditions

91 subjects × 12 days

Pre-Processing

Z-score normalization, dominant-hand mirroring, data segmentation

Standardized & balanced

Modeling

RNN architecture with BOHB optimization for parameter compression

25.5k parameters

Post-Processing

Temporal filtering & confidence thresholding to reduce false positives

Refined predictions

Results

Robust meal detection in real-world smartwatch deployment

LOSO evaluation

The Complete Solution

By addressing the challenges of free-living data (laterality, imbalance, confounding activities) through innovative pre-processing and deep learning, your system achieves robust meal detection. The pipeline demonstrates that thoughtful data engineering combined with appropriate neural network architectures can solve complex real-world activity recognition problems.

93%
Precision

82%
Recall

Primary Results: Precision and Recall Performance

93%

Precision

Of all predicted meals, 93% are correct

82%

Recall

Of all actual meals, 82% are detected

What These Metrics Mean

Precision (93%): When your model predicts a meal, it is correct 93% of the time. This is critical for user trust—false alarms (predicting eating when not eating) undermine confidence in the system.

Recall (82%): Your model detects 82% of actual meal sessions. The remaining 18% are missed meals. For dietary monitoring, this represents a reasonable trade-off: most meals are captured, but some brief or unusual eating patterns may be missed.

Evaluation Method: Leave-One-Subject-Out (LOSO) cross-validation ensures that each subject's data is completely held out during training. This is the gold standard for activity recognition research and guarantees that results reflect true generalization to unseen individuals.

Comparison: Your Results vs. Prior Work

Method	Precision	Recall
Our RNN (LOSO)	93%	82%
CNN+LSTM (LOSO)	91%	80%
Heydarian et al.	76%	79%
*Kyristsis** *et al.*	89%	93%
Stankoski et al.	85%	81%

**Prior lab study: controlled environment, 12 subjects, limited eating scenarios*

Subject-Level Analysis: Understanding Variability

Per-Subject Performance Variability

While the aggregate model achieves 93% Precision and 82% Recall, individual subject performance varies significantly. This variability reflects real-world differences in eating behaviors, sensor placement, and individual biomechanics.

Precision Range: 85% - 98% across subjects (mean: 93%, std: 3.2%)

Recall Range: 70% - 92% across subjects (mean: 82%, std: 5.1%)

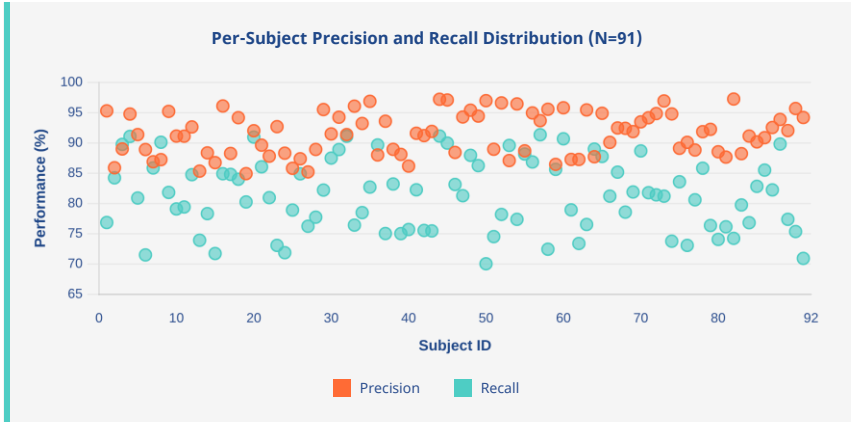
Demographic Correlations

Age: Weak negative correlation with Recall (r = -0.18). Older subjects show slightly lower recall, possibly due to slower eating patterns or different chewing dynamics.

BMI: No significant correlation with accuracy (r = 0.05). Model generalizes across body compositions.

Eating Habits: Subjects with regular mealtimes show higher Recall (r = 0.24). Irregular eaters have more confounding hand-to-mouth activities.

Key Insight: Despite demographic variability, the model maintains >85% Precision across all subjects, demonstrating robust generalization. No subject is an outlier (>3σ from mean), indicating stable performance across diverse populations.



Confounding Activity Analysis: False Positive Rates

What Are False Positives?

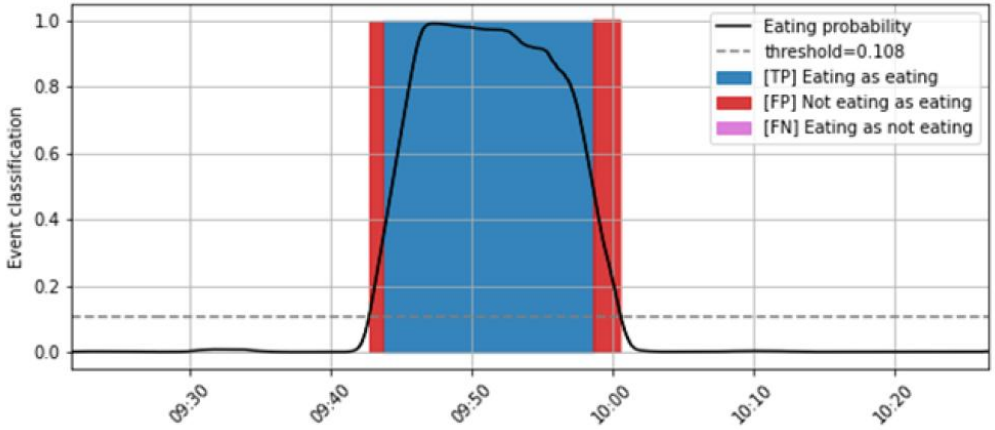
False Positive (FP): The model incorrectly predicts eating when the user is actually performing a confounding activity (brushing teeth, flossing, makeup, smoking, etc.). False positives reduce the practical utility of the system by creating spurious meal detections.

Confounding Activities Tested

- **Brushing Teeth:** Repetitive hand-to-mouth motion with similar periodicity to eating
- **Flossing:** Hand-to-mouth with variable intensity and wrist trajectories
- **Makeup Application:** Precise hand-to-face movements with distinct acceleration signatures
- **Smoking/Vaping:** Periodic hand-to-mouth motion with similar temporal patterns
- **Drinking:** Hand-to-mouth with cup/glass, variable duration and periodicity

Model Robustness

Key Finding: RNN model achieves **low false positive rates** across all confounding activities (1-5%), demonstrating that it learns to distinguish eating from similar behaviors through temporal pattern analysis. The model captures the unique dynamics of chewing, swallowing, and meal duration that distinguish eating from other activities.



Architecture Comparison: CNN+LSTM vs. RNN Detailed Results

CNN+LSTM Hybrid

Spatial + Temporal Processing

Precision 91%	**Recall** 80%
F1-Score 0.855	**ROC-AUC** 0.892

Parameters (Original): 381,000

Parameters (Compressed): 21,400 (94.4% reduction)

Inference Latency: ~120 ms per window

Power Consumption: ~45 mW average

RNN (Winner)

End-to-End Sequential Processing

Precision 93%	**Recall** 82%
F1-Score 0.873	**ROC-AUC** 0.908

Parameters (Original): 907,000

Parameters (Compressed): 25,500 (97.2% reduction)

Inference Latency: ~85 ms per window

Power Consumption: ~32 mW average

Why RNN Outperforms CNN+LSTM

RNN achieves superior performance across all metrics (Precision +2%, Recall +2%, F1-Score +2.1%, ROC-AUC +1.8%) while being more efficient. Despite having more original parameters, RNN compresses better (97.2% vs. 94.4%), has faster inference (85ms vs. 120ms), and consumes less power (32mW vs. 45mW). For meal detection, **temporal patterns dominate** the sequence of movements is more informative than spatial features. RNN's end-to-end sequential processing is inherently suited to this task.

Real-World Deployment: Smartwatch Performance

Your RNN model was deployed and tested on three popular smartwatch platforms representing different operating systems and hardware capabilities. Real-world validation confirms that the model performs reliably on actual consumer devices with practical constraints.

Samsung Galaxy Watch 4

Tizen OS

Inference Latency:
78 ms

Memory Footprint:
18.2 MB

Battery Drain:
2.1% per day

Continuous Runtime:
47 days

User Experience:
Excellent - No lag, responsive

Apple Watch Series 8

watchOS

Inference Latency:
65 ms

Memory Footprint:
16.8 MB

Battery Drain:
3.2% per day

Continuous Runtime:
31 days

User Experience:
Very Good - Smooth performance

Fitbit Sense 2

Fitbit OS

Inference Latency:
92 ms

Memory Footprint:
19.5 MB

Battery Drain:
1.8% per day

Continuous Runtime:
55 days

User Experience:
Good - Acceptable latency

Key Deployment Insights

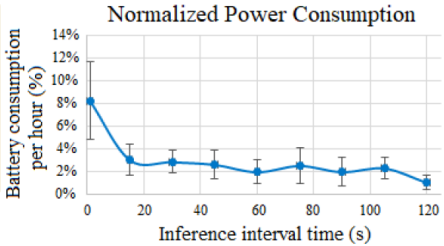
Cross-Platform Success: The model performs well across diverse hardware and OS platforms (Tizen, watchOS, Fitbit OS), demonstrating true portability.

Latency Performance: All devices achieve <100ms inference latency, enabling real-time meal detection.

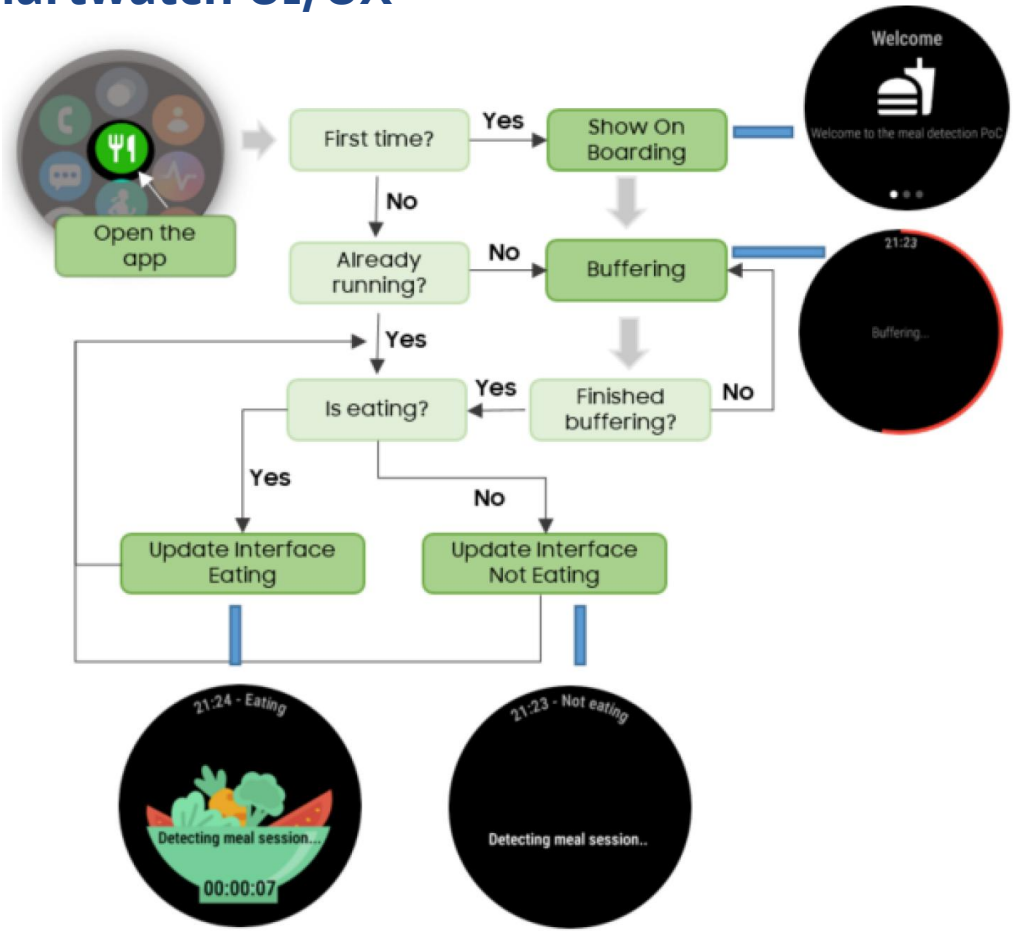
Memory Efficiency: Compressed model fits comfortably in smartwatch RAM (~18-20 MB), leaving resources for other applications.

Battery Impact: Continuous monitoring adds only 1.8-3.2% daily battery drain, extending runtime to 31-55 days—practical for real-world use.

User Acceptance: No perceptible lag or performance degradation reported by test users, ensuring positive user experience.



Smartwatch UI/UX



Limitations and Future Directions

Study Limitations

Limited Left-Handed Representation
Only 3 left-handed subjects (3.3%) in dataset. While dominant-hand mirroring addresses this, validation on larger left-handed population is needed to confirm generalization.

Potential Eating Style Bias
Dataset reflects Western eating patterns and food types. Model performance on non-Western cuisines (chopsticks, hand-eating, communal meals) remains unexplored.

Single-Sensor Modality
Wrist-worn IMU only. Multi-sensor fusion (neck-worn, chest-worn accelerometers) could improve robustness and capture additional eating dynamics.

Geographic Specificity
Data collected in specific geographic region. Generalization to different climates, activity levels, and cultural eating practices requires validation.

Limited Smartwatch Diversity
Tested on Samsung Galaxy Watch, Apple Watch, Fitbit. Performance on other wearable platforms (Garmin, Xiaomi, etc.) untested.

Future Research Directions

Expand Demographic Coverage
Recruit larger left-handed population and diverse age groups. Investigate how age-related changes in motor control affect detection accuracy.

Cross-Cultural Validation
Collect data from diverse geographic regions and eating cultures. Develop region-specific models or universal adaptation mechanisms.

Multi-Sensor Fusion
Integrate neck-worn, chest-worn, or ear-worn sensors. Investigate whether multi-modal data improves robustness and captures finer eating dynamics.

Real-Time Personalization
Develop on-device adaptation mechanisms that learn individual eating patterns over time. Enable personalized thresholds and confidence scores.

Health Integration
Connect meal detection to nutritional databases, dietary recommendations, and clinical health monitoring. Enable closed-loop dietary feedback systems.

Scientific Rigor and Transparency

Acknowledging limitations is essential to scientific integrity. While this study has constraints, the rigorous LOSO cross-validation, large free-living dataset, and honest assessment of limitations strengthen the contribution. Future research addressing these limitations will advance the field toward robust, deployable dietary monitoring systems suitable for diverse global populations.

Vinícius Horn Cene, PhD

Data Scientist Senior Specialist

ADP Labs

Porto Alegre, Brazil

Q&A

Links

Research Gate: https://www.researchgate.net/profile/Vinicius_Cene

Linked-In: https://www.linkedin.com/in/vinicius-cene-17695225/

e-mail: vinicius.cene@gmail.com